Parallel Programming · Winter term 2026/27

Exercise 2



Task 1 · `hello.c`

The following listing shows a simple Hello-World program for OpenMP:

```c
#include <stdio.h>
#include <omp.h>

int main() {
    #pragma omp parallel
    {
        printf("Hello World!\n");
    }
    return 0;
}
```

Compile and execute it. Investigate what happens and explain why.

Compile:

```
gcc -o hello -fopenmp hello.c
```

Run:

```
./hello
```

Task 2 · `count3s.c`

Implement all `count3s` variants discussed in Lecture 2 as different methods within a single program. Add for each variant a proper time measurement using `omp_get_wtime()`. Run the program on the lab PC using varying numbers of threads and compare the results to investigate their parallel performance. Compare your results with those presented in the lecture slides and evaluate to what extent they can be reproduced. Finally, analyze which program parameters have an impact on performance.

Task 3 · `matrix_vector.c` (optional)

This task serves as preparation for the next weeks. Implement a *sequential* matrix–vector multiplication in `matrix_vector.c`. The number of rows and columns of the matrix should be provided as command-line arguments. Initialize all elements of the matrix and the vector with random values using a fixed seed of 42. This task is optional.

Task 4 · Free AI Coding Agents (optional)

Attention

AI tools are recommended, but the work must be your own. We recommend using AI tools such as HAWKI or AI coding agents for the exercises, e.g., to have your code explained and reviewed. They are **not necessary**: you can complete all tasks and pass the exam without

them. You remain fully responsible for everything you hand in or present: every line of code, every measurement, and every statement, including those an AI tool suggested. AI output can be wrong even when it sounds convincing, so check it and use only what you understand. **AI tools are not available in the oral exam**, so you must be able to explain code and results on your own.

These rules follow the principles of good research practice. The German Research Foundation (DFG) states that generative AI may be used, but that its use does not relieve anyone of their responsibility for the content, that it must be disclosed (which tool, for what purpose, to what extent), and that only humans can be authors: `https://www.dfg.de/de/grundlagen-themen/digitale-themen/ki`.

Free AI Coding Agents

AI coding agents run in your terminal, read the files of your project, and can explain, review, and edit code. The following options are free of charge:

- **OpenCode** (**our recommendation**), `https://opencode.ai`: open-source agent for macOS and Linux (Windows via WSL). It provides free models via *OpenCode Zen*. The free model *Big Pickle* works well for this course and needs no credit card.
- **GitHub Education**, `https://education.github.com/pack`: verified students get the Student Developer Pack and the free *GitHub Copilot Student* plan (code completions, limited chat and agent usage, automatic model selection). Copilot can be used in VS Code, in the GitHub Copilot CLI, and in OpenCode (`/connect`, then *GitHub Copilot*). Verification requires your university e-mail address and a proof of enrollment and may take several days.

Free offers and free models change often (status: October 2026). Free models may log your prompts and code and use them for training: **never enter personal data, passwords, or other confidential information.**

In Exercise 1, you could try HAWKI, a chat that only sees what you paste into it. An AI coding agent goes further: it runs in your terminal and works directly on the files of your project. Set up a free AI coding agent and use it as a *tutor* for your own code, not as a code generator. This task is optional.

Setup

Install OpenCode on your own laptop and start it in your exercise folder:

```
curl -fsSL https://opencode.ai/install | bash   # or: npm install -g opencode-ai
cd paraprog/exercise02                          # your exercise folder
opencode
```

Inside OpenCode, type `/models` and select *Big Pickle*. Press `Tab` to switch to the *Plan* agent: it only reads and explains your code and does not modify your files. On Windows, use WSL. You can also install OpenCode on the lab PCs, but, like local files, it may be gone after a reboot.

Tasks

a) Ask the agent to explain the `count3s` variants in your `count3s.c`. Compare its explanation with the variants from Lecture 2. Where they differ, find out who is right.
b) Ask the agent to review `count3s.c` for correctness (e.g., race conditions, shared variables) and performance (e.g., false sharing). Check every claim yourself.
c) Ask the agent why your measurements from Task 2 differ from those on the lecture slides. Check its answer against your own measurements.